Genomic Data Science · DATA405/BIOL480 · Fall 2026 · Howard University

# Lab 1 · Python and R on real chr22 data

Wednesday Oct 7, 2026 · 5:10–6:40 pm · 1.5 hours · Instructor: Jinrui Xu

Name: _____ Lab partner (optional): _____

## Before you start

- Open the course website in any browser (Chrome, Edge, Firefox, Safari) on Windows, Mac or Chromebook and go to Lab 1. Nothing is installed; Python and R run inside the page. The first cell in each language takes 10–30 s to start.
- Do every exercise in **both** languages. Predict the output before you press Run. Use the Hint before the Solution.
- ChatGPT is allowed (Exercise 6 requires it). Copilot-style autocomplete is not used in this lab: the exercises are the exact lines it would type for you.
- Write your answers to Q1–Q7 on this handout or in your lab report. Report due Sunday, Oct 18, 11:59 pm on Canvas (one page is enough).

### Data used today (all real)

| File | What | Used in |
|---|---|---|
| `COMT_cds.fa` | coding sequence of COMT (chr22), 816 nt, GRCh38 | Ex 2 |
| `reads_5k.fastq` | 5,000 real ENCODE RNA-seq reads, 100 nt | Ex 3 |
| `chr22_genes.tsv` | 1,747 GENCODE genes on chr22 | Ex 4 |
| `chr22_expression.tsv` | RSEM expression of those genes in one sample (BIOL419 Lab 4 pipeline) | Ex 4 |
| `chr22_mismatches.tsv` | read-vs-reference mismatches from that sample | Ex 4 |
| `airway_counts.tsv`, `airway_metadata.tsv` | 8-sample RNA-seq count matrix, dexamethasone vs control (Himes 2014) | Ex 5 |

## Exercise 1 · Hello, two languages (10 min)

Run both cells, then change the gene name and the number and run again.

**Python**

```
gene = "COMT"
n_aa = 271
print(gene, "encodes", n_aa, "amino acids")
print("That needs", n_aa * 3, "nt + a stop")
```

**R**

```
gene <- "COMT"
n_aa <- 271
cat(gene, "encodes", n_aa, "amino acids\n")
cat("That needs", n_aa * 3, "nt + a stop\n")
```

**Q1.** In the R cell, what happens if you write n_aa = 271 instead of n_aa <- 271? And in Python, n_aa <- 271? Try both; one sentence.

## Exercise 2 · DNA as a string: COMT and rs4680 (20 min)

Run the setup cell first (defines read_fasta and reads the sequence into cds).

**Python**

```
def read_fasta(path):
    seq = []
    for line in open(path):
        if not line.startswith(">"):
            seq.append(line.strip())
    return "".join(seq)

cds = read_fasta("data/COMT_cds.fa")
print(len(cds), "nt")
```

**R**

```
read_fasta <- function(path) {
  lines <- readLines(path)
  paste(lines[!startsWith(lines, ">")],
        collapse = "")
}

cds <- read_fasta("data/COMT_cds.fa")
cat(nchar(cds), "nt\n")
```

### 2a · GC content — fill in the blanks

**Python**

```
g = cds.count("G")
c = ______
gc = (g + c) / ______
print(round(gc * 100, 1), "%")
```

**R**

```
bases <- strsplit(cds, "")[[1]]
gc <- mean(bases %in% ______)
cat(round(gc * 100, 1), "%\n")
```

**Checkpoint** Both languages must print 61.8 %. (The human genome averages about 41 %.)

### 2b · Reverse complement

**Python**

```
complement = {"A": "T", "T": "A",
              "G": "C", "C": "G"}
rc = "".join(complement[b] for b in ______)
print(rc[:20])
```

**R**

```
complement <- c(A = "T", T = "A",
                G = "C", C = "G")
rc <- paste(______(complement[bases]),
            collapse = "")
cat(substr(rc, 1, 20), "\n")
```

**Checkpoint** The reverse complement must start with TCA, because the CDS ends with the stop codon TGA.

### 2c · Translate, then mutate codon 158

Run the setup cell on the website (codon table + translate). Then complete:

**Python**

```
i = (158 - 1) * 3
codon = cds[i:i+3]
print("codon 158:", codon, "->",
codon_table[codon])
mutant = cds[:i] + ______ + cds[i+1:]
print("mutant   :", mutant[i:i+3], "->",
      translate(mutant)[______])
```

**R**

```
i <- (158 - 1) * 3 + 1
codon <- substr(cds, i, i + 2)
cat("codon 158:", codon, "->",
codon_table[codon], "\n")
mutant <- cds
substr(mutant, i, i) <- ______
cat("mutant   :", substr(mutant, i, i + 2),
"->",
    substr(translate(mutant), ______,
______), "\n")
```

**Checkpoint** Protein starts MPEAPPLLLA, 271 aa. Codon 158: GTG (V) → ATG (M). This is Val158Met (rs4680).

**Q2.** Delete the letter instead of substituting it. How many amino acids still match before the sequence goes wrong, and where is the first stop codon? Why is a deletion usually worse than a substitution?

## Exercise 3 · Reads: FASTQ and quality (15 min)

Run the setup cell (reads all 5,000 records into seqs and quals).

**Python**

```
seqs, quals = [], []
with open("data/reads_5k.fastq") as f:
    while True:
        name = f.readline()
        if not name:
            break
        seqs.append(f.readline().strip())
        f.readline()
        quals.append(f.readline().strip())
```

**R**

```
lines <- readLines("data/reads_5k.fastq")
n <- length(lines)
seqs  <- lines[seq(2, n, by = 4)]
quals <- lines[seq(4, n, by = 4)]
```

### 3a · Decode quality and plot the mean per position

**Python**

```
L = len(seqs[0])
mean_q = []
for i in range(L):
    tot = sum(ord(q[i]) - ______ for q in
quals)
    mean_q.append(tot / len(quals))
fig, ax = plt.subplots(figsize=(4, 2.6))
ax.plot(range(1, L + 1), mean_q)
```

**R**

```
Q <- t(sapply(quals,
       function(s) utf8ToInt(s) - ______))
mean_q <- colMeans(Q)
plot(mean_q, type = "l",
     xlab = "position", ylab = "mean Q")
```

**Checkpoint** Mean Q ≈ 33 at position 1, ≈ 37 in the middle, ≈ 35 at position 100.

### 3b · Count reads by content

**Python**

```
n_N = sum("N" in s for s in seqs)
n_polyA = sum(______ in s for s in seqs)
gc = [(s.count("G") + s.count("C")) / len(s)
      for s in seqs]
print(n_N, n_polyA, round(sum(gc)/len(gc),
3))
```

**R**

```
n_N <- sum(grepl("N", seqs))
n_polyA <- sum(grepl(______, seqs))
gc <- sapply(strsplit(seqs, ""),
  function(s) mean(s %in% c("G", "C")))
cat(n_N, n_polyA, round(mean(gc), 3), "\n")
```

**Checkpoint** 6 reads contain N; 63 contain 15 A's in a row; mean GC ≈ 0.474.

**Q3.** The reads are ~47 % GC, the genome is 41 %, COMT is 62 %. Give one biological reason RNA-seq reads are GC-richer than the genome.

## Exercise 4 · Tables: genes, expression, mismatches (20 min)

**Python**

```
import pandas as pd
genes = pd.read_csv("data/chr22_genes.tsv",
sep="\t")
expr  =
pd.read_csv("data/chr22_expression.tsv",
sep="\t")
mm    =
pd.read_csv("data/chr22_mismatches.tsv",
sep="\t")
```

**R**

```
genes <- read.delim("data/chr22_genes.tsv")
expr  <-
read.delim("data/chr22_expression.tsv")
mm    <-
read.delim("data/chr22_mismatches.tsv")
```

### 4a · Filter, sort, count

How many protein-coding genes are on the minus strand? Which three protein-coding genes are longest?

**Python**

```
pc = genes[genes["gene_type"] ==
"protein_coding"]
print((pc["strand"] == ______).sum())
print(pc.sort_values("length_bp",
      ascending=______)[["gene_name",
"length_bp"]].head(3))
```

**R**

```
pc <- genes[genes$gene_type ==
"protein_coding", ]
sum(pc$strand == ______)
head(pc[order(______pc$length_bp),
        c("gene_name", "length_bp")], 3)
```

### 4b · Join and summarize

Join genes to expr by gene_id; for each gene_type compute the fraction of genes with TPM > 1.

**Python**

```
m = genes.merge(expr[["gene_id", "TPM"]],
on=______)
frac = m.groupby("gene_type")["TPM"] \
        .apply(lambda s: (s > 1).mean())
print(frac.sort_values(ascending=False).head
(5))
```

**R**

```
m <- merge(genes, expr[, c("gene_id",
"TPM")],
           by = ______)
frac <- tapply(m$TPM > 1, m$gene_type, mean)
head(sort(frac, decreasing = TRUE), 5)
```

> **Checkpoint** About 47 % of protein-coding genes have TPM > 1. The top five expressed genes are immunoglobulin genes and pseudogenes.

**Q4.** What cell type do the top-expressed genes suggest? Why can a pseudogene (ACTBP15) show a high TPM even though it is not transcribed? (Hint: what is ACTBP15 a copy of, and what was in the reference?)

---

### 4c · Mismatches: which look like real variants?

Filter to depth ≥ 10 and allele fraction between 0.3 and 0.7 (what a heterozygous SNP looks like). Then tabulate the most common substitution types.

**Python**

```
het = mm[(mm["depth"] >= 10) &
         (mm["alt_fraction"] >= ______) &
         (mm["alt_fraction"] <= ______)]
print(len(het))
subst = mm["ref"] + ">" + mm["alt"]
print(subst.value_counts().head(4))
```

**R**

```
het <- mm[mm$depth >= 10 &
          mm$alt_fraction >= ______ &
          mm$alt_fraction <= ______, ]
nrow(het)
subst <- paste0(mm$ref, ">", mm$alt)
head(sort(table(subst), decreasing = TRUE),
4)
```

**Checkpoint** Only 8 of 1,097 sites look heterozygous. The two most common substitutions are T>C and A>G.

**Q5.** A>G (and its mirror T>C) dominating RNA-seq mismatches is a known biological signal, not an error. What is it, and why does DNA sequencing not show it?

---

## Exercise 5 · Statistics on counts: the t-test and its limits (15 min)

**Python**

```
import numpy as np
from scipy import stats
cts  = pd.read_csv("data/airway_counts.tsv",
                   sep="\t", index_col=0)
meta =
pd.read_csv("data/airway_metadata.tsv",
sep="\t")
logc = np.log2(cts + 1)
ctrl = list(meta["id"][meta["dex"] ==
"control"])
trt  = list(meta["id"][meta["dex"] ==
"treated"])
```

**R**

```
cts  <- read.delim("data/airway_counts.tsv",
                   row.names = 1)
meta <-
read.delim("data/airway_metadata.tsv")
logc <- log2(cts + 1)
ctrl <- meta$id[meta$dex == "control"]
trt  <- meta$id[meta$dex == "treated"]
```

### 5a · One gene (ENSG00000152583, SPARCL1): boxplot and t-test

**Python**

```
g = "ENSG00000152583"
a = logc.loc[g, ctrl]
b = logc.loc[g, trt]
fig, ax = plt.subplots(figsize=(3, 3))
ax.boxplot([a, b], labels=["control",
"dex"])
print(stats.ttest_ind(______, ______))
```

**R**

```
g <- "ENSG00000152583"
a <- unlist(logc[g, ctrl])
b <- unlist(logc[g, trt])
boxplot(list(control = a, dex = b))
t.test(______, ______)
```

### 5b · The 19,271-tests trap

Python

```
p = stats.ttest_ind(logc[trt], logc[ctrl],
axis=1).pvalue
print("p < 0.05:", (p < ______).sum(), "of",
len(p))
rng = np.random.default_rng(1)
cols = list(cts.columns); rng.shuffle(cols)
p_shuf = stats.ttest_ind(logc[cols[:4]],
                         logc[cols[4:]],
axis=1).pvalue
print("shuffled:", (p_shuf < 0.05).sum())
```

R

```
tt <- function(a, b) tryCatch(t.test(a, b)
$p.value,
                              error =
function(e) NA)
p <- apply(logc, 1, function(x) tt(x[trt],
x[ctrl]))
cat("p < 0.05:", sum(p < ______, na.rm =
TRUE), "\n")
set.seed(2)
cols <- sample(colnames(cts))
p_shuf <- apply(logc, 1,
  function(x) tt(x[cols[1:4]],
x[cols[5:8]]))
cat("shuffled:", sum(p_shuf < 0.05, na.rm =
TRUE), "\n")
```

> **Checkpoint** With random labels, hundreds to over a thousand genes still pass p < 0.05. Now apply Benjamini–Hochberg (p.adjust(p, "BH") in R) — how many survive at FDR 0.05?

**Q6.** Two reasons the t-test is the wrong tool here: what does it assume about the distribution of counts, and what does it ignore about the four donors?

---

## Exercise 6 · The apprentice: catch ChatGPT's bug (10 min)

- Ask ChatGPT (TRACE prompt): "Write a Python (then R) function reverse_complement(dna) for DNA sequences that may contain N."
- Paste its function into the Exercise 6 cell on the website and run the three tests: ATGC → GCAT, AAANTT → AANTTT, atgc → GCAT.
- If any test fails, fix the function yourself. Paste the fixed version (both languages) in your report with the ChatGPT thread link.

**Q7.** The typical first answer fails two of the three tests. Name the two bugs. Run it in both languages: in which language does it crash loudly, and in which does it silently give a wrong answer? Which is worse, and why?

---

## Lab report (due Sunday, Oct 18, 11:59 pm)

Answers to Q1–Q7 · the fixed reverse_complement in both languages · one figure from Exercise 3 or 5 (from the local setup, or right-click → save image from the website). One page is enough.

*Data provenance and local setup (Windows/Mac): see the Data and Setup pages of the course website.*